

LABORATORY OBSERVATION / RECORD

Course	Full Stack Web Development	Experiment No.	Task 5
Course Code	23CM4121	Student Name	R Abhiram
Reg. No.	A24126552114	Date	21-09-2026

1. TITLE

File Management Application Using Node.js (fs and readline Modules)

2. AIM

To create a Node.js console application that accepts a filename and content from the user, creates and writes the file, reads and displays its contents, appends additional content to it, and finally displays the updated file contents — using the core **fs** and **readline** modules.

3. OBJECTIVE

The objectives of this experiment are:

- To take user input from the console using the readline module.
- To create and write to a file asynchronously using `fs.writeFile()`.
- To read the contents of a file using `fs.readFile()`.
- To append additional content to an existing file using `fs.appendFile()`.
- To handle asynchronous file operations correctly using callback functions.
- To display the initial and final file contents on the console without any external editor.

4. THEORY

Node.js is a JavaScript runtime built on the V8 engine that lets JavaScript run outside the browser and provides built-in (core) modules for tasks such as file handling and console I/O. This program combines the **fs** module for file operations with the **readline** module for interactive console input.

- **fs module:** Node's core module for creating, reading, writing, appending to and deleting files on disk.
- **readline module:** reads a readable stream (such as `process.stdin`) line by line, commonly used to collect input from the terminal.
- **readline.createInterface():** binds the readline interface to `process.stdin` (input) and `process.stdout` (output).
- **rl.question(prompt, callback):** prints the prompt, waits for the user to type a line, and passes it to the callback.
- **fs.writeFile(path, data, callback):** creates a new file or overwrites an existing one with the given data.
- **fs.readFile(path, encoding, callback):** asynchronously reads the entire file and returns its contents as a string.
- **fs.appendFile(path, data, callback):** adds data to the end of an existing file without erasing what is already there.
- **Callback functions:** Node's file system calls are non-blocking (asynchronous), so a callback executes only after the operation completes.
- **rl.close():** ends the readline session, allowing the Node.js process to exit once all I/O is finished.

Flow: filename entered → initial content entered → file created/written → file read and displayed → append content entered → content appended → final file read and displayed → readline closed

5. ALGORITHM / PROCEDURE

1. Import the core fs and readline modules.
2. Create a readline interface bound to process.stdin and process.stdout.
3. Prompt the user to enter a filename using rl.question().
4. Prompt the user to enter the initial content for the file.
5. Use fs.writeFile() to create the file and write the initial content into it.
6. Use fs.readFile() to read back and print the initial file contents.
7. Prompt the user to enter content to append to the file.
8. Use fs.appendFile() to add the new content to the end of the file.
9. Use fs.readFile() again to read and print the final file contents.
10. Call rl.close() to end the readline session and let the program terminate.
11. Run the program with node app.js and repeat with different filenames and content to verify the behaviour.

6. PROGRAM

app.js

```
// Node.js File Management Application
const fs = require('fs');
const readline = require('readline');

// Create readline interface for user input
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

// Step 1: Ask for filename
rl.question('Enter the filename: ', (filename) => {

  // Step 2: Ask for initial content
  rl.question('Enter the initial content: ', (content) => {

    // Create/Write the file
    fs.writeFile(filename, content, (err) => {
      if (err) throw err;
      console.log(`\n File '${filename}' created and written successfully.`);

      // Read the file contents
      fs.readFile(filename, 'utf8', (err, data) => {
        if (err) throw err;
        console.log('\n Initial File Contents:');
        console.log(data);

        // Step 3: Ask for content to append
        rl.question('\nEnter content to append: ', (appendContent) => {
          fs.appendFile(filename, appendContent, (err) => {
            if (err) throw err;
            console.log('\n Content appended successfully.');
```

7. CODE EXPLANATION

Code	Explanation
<code>require('fs')</code>	Imports Node's core file system module for file operations.
<code>require('readline')</code>	Imports the readline module used to read console input line by line.
<code>readline.createInterface()</code>	Creates an interface that connects the terminal's input and output streams.
<code>rl.question(prompt, cb)</code>	Displays a prompt and passes the line typed by the user to the callback.
<code>fs.writeFile(filename, content, cb)</code>	Creates the file (or overwrites it if it exists) and writes the given content.
<code>fs.readFile(filename, 'utf8', cb)</code>	Reads the complete contents of the file as a UTF-8 string.
<code>fs.appendFile(filename, content, cb)</code>	Adds the new content to the end of the existing file without deleting prior content.
<code>if (err) throw err;</code>	Basic error handling — stops execution and reports the error if a file operation fails.
<code>rl.close();</code>	Closes the readline interface once all questions have been answered, allowing the process to exit.

8. TEST CASES AND EXECUTION DATA

Each test case was executed by running `node app.js` in the terminal, entering the values below at each prompt, and observing the console output and resulting file.

Test Case	Filename	Initial Content	Append Content	Expected Final Content	Status
TC-01	notes.txt	Hello World, this is Node.js file handling.	This line is appended dynamically to the file.	Hello World, this is Node.js file handling.This line is appended...	Pass
TC-02	data.txt	Node.js File Handling Demo	Second line added via append operation.	Node.js File Handling DemoSecond line added via append operation.	Pass
TC-03	report.txt	Initial project report content.	(space) Updated on review.	Initial project report content. Updated on review.	Pass
TC-04	notes.txt (reused)	This is completely new content.	(space) - extra note added.	This is completely new content. - extra note added. (old content overwritten)	Pass

9. OUTPUT

Output 1: TC-01 – notes.txt

```
$ node app.js
Enter the filename: notes.txt
Enter the initial content: Hello World, this is Node.js file handling.

File 'notes.txt' created and written successfully.

Initial File Contents:
Hello World, this is Node.js file handling.

Enter content to append: This line is appended dynamically to the file.

Content appended successfully.

Final File Contents:
Hello World, this is Node.js file handling.This line is appended dynamically to the file.
```

The file was created, written, read back, appended to and re-read — all confirmed on the console.

Output 2: TC-02 – data.txt

```
$ node app.js
Enter the filename: data.txt
Enter the initial content: Node.js File Handling Demo

File 'data.txt' created and written successfully.

Initial File Contents:
Node.js File Handling Demo

Enter content to append: Second line added via append operation.

Content appended successfully.

Final File Contents:
Node.js File Handling DemoSecond line added via append operation.
```

Appended text is joined immediately after the existing content since no newline was typed by the user.

Output 3: TC-03 – report.txt

```
$ node app.js
Enter the filename: report.txt
Enter the initial content: Initial project report content.

File 'report.txt' created and written successfully.

Initial File Contents:
Initial project report content.

Enter content to append: Updated on review.

Content appended successfully.

Final File Contents:
Initial project report content. Updated on review.
```

Output 4: TC-04 – reusing an existing filename (notes.txt)

```
$ node app.js
Enter the filename: notes.txt
Enter the initial content: This is completely new content.

File 'notes.txt' created and written successfully.

Initial File Contents:
This is completely new content.

Enter content to append: - extra note added.

Content appended successfully.

Final File Contents:
This is completely new content. - extra note added.
```

fs.writeFile() overwrote the previous notes.txt entirely, confirming that it replaces rather than merges existing content.

10. RESULT

Thus, a file management console application was successfully developed using Node.js. The fs and readline core modules were used to take user input, create and write a file, read and display its contents, append additional content, and display the final result — all handled asynchronously through callback functions. The application was verified using multiple test cases.